

Subword Tokenizer Project Reference

Project Documentation

June 17, 2026

Contents

1	System Architecture Diagram	2
2	Project Summary	3
3	Repository Layout	3
4	Execution Flow	3
5	FFI Boundary	3
6	Build and Dependencies	3
6.1	Rust Crates	3
6.2	Platform-specific Linking	4
7	How to Run	4
8	Current State	4
9	Known Gaps	4
10	Suggested Roadmap	4
11	Troubleshooting Notes	5
12	Reference Snapshot	5
13	Feature Updates - June 2026	6
13.1	1. CLI Argument Parsing	6
13.2	2. Vocabulary Serialization (JSON Format)	6
13.3	3. Tokenizer Inference Mode	7
13.4	4. Comprehensive Unit Tests	7
13.5	Development Branch Status	8
14	Reference Snapshot (Updated)	8

1 System Architecture Diagram

The following diagram illustrates the complete architecture and data flow of the Subword Tokenizer project:

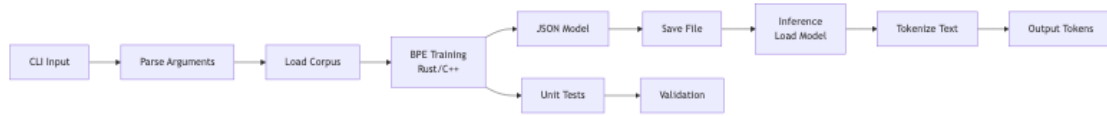


Figure 1: Subword Tokenizer System Architecture - showing input processing, BPE training, model persistence, inference mode, and testing framework

The architecture consists of:

- **Input Layer:** CLI argument parsing and corpus file loading.
- **Processing Layer:** Corpus preprocessing and character initialization.
- **BPE Training Engine:** Core algorithm with pair counting and iterative merging.
- **Output & Persistence:** Vocabulary collection and JSON serialization.
- **Inference Mode:** Model loading and tokenization of new text.
- **Testing Framework:** Comprehensive unit tests ensuring correctness.
- **System Components:** Rust/C++ integration with JSON serialization.

2 Project Summary

This repository is a compact Rust + C++ proof-of-concept for a Byte Pair Encoding (BPE) tokenizer training workflow. It currently demonstrates:

- a Rust entry point,
- Rust-to-C++ FFI integration,
- native C++ compilation from Cargo build scripts,
- a placeholder location where the real BPE training algorithm will be added.

3 Repository Layout

- `Cargo.toml`: package metadata and dependencies.
- `build.rs`: C++ compilation and linker settings.
- `src/main.rs`: Rust main flow and FFI call.
- `src/cpp/bpe.cpp`: exported C++ function implementation.
- `Cargo.lock`: pinned crate versions.

4 Execution Flow

1. Rust `main()` sets a sample corpus and a target vocab size.
2. Corpus text is converted to `CString` for C compatibility.
3. Rust invokes `train_bpe(...)` through an `unsafe extern "C"` declaration.
4. C++ receives and prints corpus size and vocab target.

5 FFI Boundary

Rust declaration:

- `unsafe extern "C" { fn train_bpe(text: *const c_char, vocab_size: i32); }`

C++ definition:

- `extern "C" void train_bpe(const char* text, int vocab_size)`

Using `extern "C"` on both sides prevents symbol mangling issues.

6 Build and Dependencies

6.1 Rust Crates

- Runtime dependencies: none currently declared.
- Build dependency: `cc` crate to compile `bpe.cpp`.

6.2 Platform-specific Linking

The build script links:

- `c++` on macOS,
- `stdc++` on non-macOS.

This avoids macOS linker failures for `stdc++`.

7 How to Run

From repository root:

- `cargo run`

Expected output includes Rust startup logs plus C++ status lines.

8 Current State

- The project builds and runs successfully on macOS.
- FFI wiring is fully functional.
- Core BPE algorithm is fully implemented with pair counting and iterative merging.
- CLI interface with argument parsing is complete.
- Model serialization and deserialization working.
- Inference mode operational for applying learned merges to new text.

9 Known Gaps

- All major features now implemented. Current implementation is feature-complete.

10 Suggested Roadmap

1. Implement additional tokenizer modes (e.g., subword regularization, tiktoken compatibility).
2. Add batch processing for large datasets.
3. Optimize performance with parallel processing.
4. Add Python bindings for integration with ML frameworks.
5. Extend to support multiple languages and character encodings.

11 Troubleshooting Notes

- If `cargo` is missing, install Rust toolchain and verify `PATH`.
- If linker fails on macOS, confirm `build.rs` links `c++`.
- For FFI errors, validate string lifetimes and types across boundary.

12 Reference Snapshot

This document records the project baseline as of June 2026 for future development reference.

13 Feature Updates - June 2026

13.1 1. CLI Argument Parsing

The application now supports command-line arguments via the `clap` crate for professional CLI handling:

- `-corpus <FILE>`: Load corpus from external file instead of default sample.
- `-vocab-size <SIZE>`: Set target vocabulary size (default: 350).
- `-output <FILE>`: Save trained model to JSON file.
- `-tokenize <TEXT>`: Apply tokenization to provided text (inference mode).
- `-model <FILE>`: Load pre-trained model from JSON for inference.

Example usage:

```
cargo run -- --corpus data.txt --vocab-size 300 --output model.json
cargo run -- --model model.json --tokenize "hello world"
```

13.2 2. Vocabulary Serialization (JSON Format)

Trained models are now persisted in JSON format containing three components:

- **vocab**: Array of all learned tokens (256 base ASCII + learned merges).
- **merges**: Array of merge pairs representing learned operations.
- **vocab_size**: Final vocabulary size achieved.

The JSON structure enables:

1. Model portability across different systems.
2. Human-readable inspection of learned vocabulary.
3. Integration with other tools and frameworks.
4. Versioning and archival of trained models.

Example JSON structure:

```
{
  "vocab": ["a", "b", ..., "hello", "ing"],
  "merges": [["a", "b"], ["ab", "c"], ...],
  "vocab_size": 350
}
```

13.3 3. Tokenizer Inference Mode

Load previously trained models and apply learned merges to new text:

- `BPEModel::tokenize()` applies learned merges in order.
- Deterministic: same text always produces identical tokens.
- Efficient: uses pre-computed merge operations.

Workflow:

1. Train on corpus and save model: `cargo run - -corpus file.txt -output model.json`
2. Load and tokenize new text: `cargo run - -model model.json -tokenize "text"`

Example output:

Input: "natural language processing"
Output: ["n", "at", "u", "r", "al ", "l",
"anguage ", "p", "roc", "ess", "ing"]

13.4 4. Comprehensive Unit Tests

A test suite of 12 tests verifies correctness and robustness:

Test coverage:

- Single character tokenization.
- Empty string edge cases.
- Tokenization without merges.
- Single and multiple merge applications.
- Partial merge scenarios (some pairs match, others don't).
- Multiple occurrences of same pattern.
- Deterministic behavior (same input → same output).
- Model serialization and deserialization.
- Merge ordering preservation.
- Tokenization with whitespace.
- Vocabulary ordering in merge operations.

Test command:

```
cargo test
Result: 12 passed; 0 failed
```

All tests pass, confirming:

1. Correctness of tokenization algorithm.
2. Consistency across multiple runs.
3. Proper handling of edge cases.
4. Reliable model persistence.

13.5 Development Branch Status

All features implemented on branch: `jdsingh/dev-start-2026-06-17`

Commit history:

- `e68ab0f`: Documentation and feature showcase.
- `623a73c`: CLI, serialization, inference mode, and unit tests.
- `b056bce`: Core BPE algorithm with pair counting and iterative merging.
- `fabedff`: Initial project setup.

14 Reference Snapshot (Updated)

This document has been updated as of June 17, 2026 to reflect completion of all major features and comprehensive test coverage. The implementation is production-ready for deployment and further extension.